Gradient-Free Continual Reinforcement Learning with Environment Detection and Specialist Reuse: the Evolutionary Neuroplastic Agent

Blas Eugenio Vicco Zarate
Independent Researcher
Valencia, Spain
blasvicco@gmail.com

***Abstract*—Reinforcement learning agents trained with gradients tend to forget an environment once they adapt to a new one. We present the Evolutionary Neuroplastic Agent (ENA), a gradient-free agent for non-stationary environments that keeps a memory of specialist policies organized by the physics it detects. ENA evolves a population of small neural networks, scaling its search effort to how badly the current specialist fails, and archives proven specialists in a Hall of Fame. A sequential change detector notices when the physics change and recognizes physics seen before; for discrete control it fits per-environment linear-Gaussian dynamics models confirmed by the Predictive-Mixture CuSum, and for continuous control, where a learning population's changing behavior confounds transition-based detection, it plays a fixed wavelet-train excitation (a chirp) and reads the physics from the spectral response. Bayesian policy reuse then chooses which archived specialist to deploy, combining the detector's evidence with the observed returns. A skill ablation over 30 replicates on a non-stationary CartPole shows that the skills are complementary: together they raise test reliability from 38.5% to 83.2%. The failure-driven search contributes most, and the archive helps only when environment detection organizes it. At an equal budget of environment steps, DQN, PPO and SAC-Discrete forget an environment they had solved, while ENA retains it and reaches 83.9% test reliability over 50 replicates, against 74.5% for the best gradient-based agent, SAC-Discrete, in less wall-clock time; PPO, however, solves each new environment sooner. On continuous-control HalfCheetah, ENA retains 115.6% of its performance on the environments it trained on first over 20 replicates, against 39.2% for PPO and 34.4% for SAC, which forget the earlier environments despite reaching higher absolute returns within each one.**

***Index Terms*—Continual Reinforcement Learning, Neuroevolution, Non-Stationary Environments, Change Detection, Policy Reuse**

I. Introduction

Deep reinforcement learning (DRL) agents are usually trained and deployed under the assumption that the environment does not change. When it does, for instance when the physics a robot experiences shift between conditions, a gradient-trained policy adapts by overwriting the very weights that encoded its previous behavior. This is *catastrophic forgetting* [1]: once the original conditions return, the agent has to learn them again from scratch.

Continual reinforcement learning studies agents that keep adapting without losing what they have already learned. Nisioti et al. [2] recently showed that neuroevolution is a competitive answer: evolution strategies achieve a good balance between adapting to new tasks and remembering old ones, while genetic algorithms are the most plastic but forget more. In both cases, however, the population evolves as a whole toward the current task. Nothing tells it that the environment has changed, and nothing tells it which earlier solution to bring back when a previous environment returns.

A different strategy is to keep a repertoire of behaviors and switch between them when conditions change. Quality-Diversity methods build such repertoires [3], and Cully et al. [4] showed that a damaged robot can recover within minutes by searching a repertoire of gaits computed in advance. Our question is whether an agent can build and use such a repertoire *online*, from a non-stationary stream it is never told about, and without gradients.

We present the **Evolutionary Neuroplastic Agent (ENA)**, a gradient-free continual RL agent. ENA evolves a population of small neural-network specialists, scores each with a *Trust* value, and scales its search effort, its *Plasticity*, to how badly the current specialist is failing. Proven specialists are archived in a *Hall of Fame*. Two further skills make that archive a usable memory: *environment detection* notices when the physics change and recognizes physics it has seen before, using a domain-appropriate driver (linear-Gaussian dynamics models with a sequential change detector for discrete control; a wavelet-chirp spectral response for continuous control, where a learning population's changing behavior confounds transition-based detection); *specialist selection* uses Bayesian policy reuse to decide which archived specialist to deploy, combining the detector's evidence with the returns it observes.

Our contributions are:

- **ENA's architecture.** Environment detection with two domain-adaptive drivers: per-niche linear-Gaussian dynamics models and the Predictive-Mixture CuSum [5] for discrete control, and a fixed wavelet-train chirp whose spectral response separates physics for continuous control (Section III-F5), where a learning population's behavioral changes confound transition-model detection. Both drivers feed Bayesian policy reuse [6] adapted to an online, non-stationary setting, on top of a population governed by Trust, Plasticity and a Hall of Fame (Section III-F).

- **A skill ablation over 30 replicates** on non-stationary CartPole. Each skill's contribution depends on the others. Plasticity is the largest single contribution. The Hall of Fame helps only together with environment detection, which adds +23.5 points of test reliability without Plasticity and +24.3 with it. The combination reaches 83.2%, against 38.5% without any skill.
- **A comparison with DQN, PPO and SAC-Discrete at equal environment steps**, ENA's candidate evaluations included. The gradient-based agents forget an environment they had solved; ENA does not, and reaches a higher test reliability (83.9% against at most 74.5%, over 50 replicates) in less wall-clock time. PPO, in turn, solves each new environment sooner, and SAC-Discrete scores slightly higher on the environments it does not forget.
- **The same comparison on continuous-control HalfCheetah**, against PPO and SAC over 20 replicates. ENA retains 115.6% of its performance on the environments it trained on first, against 39.2% for PPO and 34.4% for SAC; both differences exceed 65 percentage points and are highly significant. SAC, however, achieves substantially higher absolute returns within each environment it adapts to.
- **Negative results and reproducibility.** We report the alternative Plasticity controllers we tested against the one we use, including those the data could not separate from it, and every number in this paper is generated by scripts from the released experiment outputs.

Section II reviews related work. Section III describes ENA. Section IV presents the environments, protocols and baselines, and Section V the results. Section VI concludes.

II. Related Work

A. Non-Stationary and Continual Reinforcement Learning

A neural network trained by gradient descent on one task after another degrades on the earlier tasks, a failure first described as catastrophic interference [1]. Continual reinforcement learning studies agents that must keep learning in such conditions without losing what they already know [7]. Elastic Weight Consolidation [8] slows learning on the weights that mattered for earlier tasks; CLEAR [9] replays past experience, with behavioral cloning, and does not need to know where tasks begin and end. When tasks are fundamentally incompatible, a single network cannot serve them all: OWL [10] gives each task its own policy head on a shared feature extractor and chooses between heads at test time with a multi-armed bandit. All of these methods protect or partition the weights of gradient-trained networks. ENA avoids the problem differently: an archived specialist is never trained again, so there is nothing to protect.

B. Evolutionary Reinforcement Learning and Continual Neuroevolution

Neuroevolution optimizes neural networks by mutation and selection over a population instead of by gradients [11]. Evolution strategies [12] and genetic algorithms [13] are competitive with gradient-based RL on standard benchmarks, and hybrid methods use a population to diversify the experience of a gradient-trained learner [14]. Nisioti et al. [2] compared evolution strategies and genetic algorithms with PPO, continual RL methods and population-based RL under task and environment changes. They found that evolution strategies achieve the best balance between adapting and remembering, and that genetic algorithms are the most plastic but forget more, and they related this balance to how much the neighborhoods of good solutions in weight space overlap between consecutive tasks. Earlier, evolutionary algorithms for changing optimization problems had developed remedies such as increasing diversity after a change, maintaining diversity throughout the run, and memory [15]; ENA uses these three (Section III-A). In their methods, as in standard neuroevolution, the population evolves as a whole toward the current task: changes are not detected, and earlier solutions are not stored and recalled. ENA keeps a gradient-free population but adds both, which makes it, in that sense, a continuation of their study.

C. Quality-Diversity and Repertoire-Based Adaptation

Quality-Diversity algorithms such as MAP-Elites [3] search for many diverse, high-performing solutions instead of a single optimum, and store them in an archive. Intelligent Trial and Error [4] showed that such an archive, computed offline in simulation, lets a damaged robot find a working gait within minutes by trying archived behaviors guided by a Bayesian model. TeLAPA [16] brings the same idea to continual RL, keeping per-task archives of behaviorally diverse policies so that competent alternatives remain available when a task is revisited. ENA's Hall of Fame is also an archive of specialists, but it is built online, while the environment changes, and it is organized by the physics the agent detects rather than by a behavior descriptor.

D. Change Detection and Policy Reuse

Detecting that a data stream has changed is the classic problem of sequential change detection, from Page's CUSUM [17] to Bayesian online changepoint detection [18]. The Predictive-Mixture CuSum [5] that ENA uses detects a change away from a known distribution without a model of the post-change distribution, by mixing predictors fitted on recent windows of data. In RL, RL-CD [19] learned one model of the environment per *context* and recognized contexts by which model predicts best, an idea ENA follows. MBCD [20] combined an ensemble of learned probabilistic dynamics models with a high-confidence change-point detector whose false-alarm rate is bounded, creating a new policy for each new context and reusing an earlier one when a context recurs; SWOKS [21] detects contexts with optimal-transport distances and Kolmogorov–Smirnov tests and trains one policy per context. These systems are the closest to ENA in purpose. Their dynamics models and policies are trained by gradient descent, whereas every part of ENA, its detector included, is gradient-free. To choose which stored policy to deploy, Bayesian Policy Reuse [6] maintains a belief over task types from the returns it observes. ENA adapts it

1: select the acting specialist i (ϵ-greedy on Trust)
2: play one episode, feeding every transition to the detector
3: $D \leftarrow f(R)$; $T_i \leftarrow (1 - D)\, T_i + D\, \hat{r}(R)$
4: $\mathcal{P} \leftarrow f(R)$
5: Hall of Fame: gate i, resolve its niche, archive or clone
6: evaluate $k(\mathcal{P})$ other specialists for $s(\mathcal{P})$ steps each; update their Trust
7: keep the elites; replace the rest by mutants, crossovers and fresh specialists (gladiators are never replaced)
8: **if** a change of physics is confirmed **then**
9: start a new regime: reset the detector's evidence, the input statistics and the return history H
10: **end if**

Fig. 1. One ENA training episode. At test time the weights are frozen, $\mathcal{P} = 0$, and Bayesian policy reuse chooses the acting specialist (Section III-G).

to an online, non-stationary setting and adds the dynamics models' likelihoods to the evidence (Section III-G).

III. Methodology

The Evolutionary Neuroplastic Agent (ENA) never computes a gradient. It maintains a population of N small neural-network policies, the *specialists*, each carrying a scalar *Trust* score. In every training episode one specialist acts, its return updates its Trust, a *Plasticity* value decides how much of the population is re-evaluated, the strongest specialists may be archived in a *Hall of Fame*, and the population evolves. On top of this loop, two skills make the archive usable in a non-stationary stream: *environment detection* notices when the physics change and recognizes physics it has seen before, and *specialist selection* chooses which archived specialist to deploy. Each mechanism is implemented as an interchangeable module with an "off" setting, which is what allows the ablation in Section V. Fig. 1 summarizes one training episode.

A. Inspiration

ENA borrows two strategies from nature, each through a mechanism already established for evolutionary algorithms in changing environments.

1) Variation in proportion to stress (Plasticity): Bacteria under stress increase their mutation rate. This is a regulated and transient response: variation is raised when the current genotype is failing, and kept low otherwise, because in a well-adapted organism most mutations are harmful [22]. Evolutionary computation adopted the same idea as *triggered hypermutation*, raising the mutation rate when performance drops in a non-stationary environment [23], and more generally as the control of an algorithm's parameters during the run [24]. ENA's Plasticity extends this response from the mutation rate to the whole search effort. The failure signal (1) sets how many specialists are re-evaluated (a random subset of size k), how long each is evaluated (s steps), and how far mutants move from their parents. The share of the population replaced in each generation stays constant. What scales with failure is the amount of evidence gathered before selection and the size of the steps taken after it. The reason is cost: every candidate step is an environment step, so a converged agent should spend almost none, while an agent that has just met new physics must search broadly. In addition, freshly initialized specialists enter the population in every generation, as *random immigrants* [25] do, keeping enough diversity to follow a moving optimum.

2) Consolidated memory (Hall of Fame): The complementary learning systems theory [26] holds that the brain learns quickly in the hippocampus and consolidates slowly into the neocortex, where knowledge is protected from interference by new learning. The theory was proposed in part to explain why the brain does not suffer the catastrophic interference of gradient-trained networks [1]. ENA follows the same division of labor. The population is the fast learner, overwritten every generation; the Hall of Fame is the consolidated store, entered only through a gate, never modified, and never replaced by evolution. In evolutionary computation, a *hall of fame* first kept the best individuals of past generations in competitive coevolution [27], and memory-enhanced evolutionary algorithms store good solutions and reinsert them when earlier conditions return [28]. ENA adds two things to this kind of memory: it is indexed by the physics in which each specialist succeeded (Section III-F), and recall is a Bayesian decision rather than a reinsertion (Section III-G).

B. Population and Specialists

Each specialist is a multi-layer perceptron with orthogonal initialization. Its inputs are normalized with a running mean and variance. Because those statistics keep drifting while the agent explores, a specialist's first layer is analytically rescaled to the current statistics before it acts, so its behavior is preserved without any retraining. During training the acting specialist is chosen ϵ-greedily: with probability ϵ a random non-archived specialist, otherwise the one with the highest Trust.

C. Trust and Plasticity

Both quantities derive from the same failure signal. Let H be the history of the last 100 episode returns, $P_q(H)$ its q-th percentile and $R_{\min}$ the worst return seen so far. For a return R,

$$f(R) = \begin{cases} 1 & |H| < 10 \\ 0.1 & R \geq P_{90}(H) \\ 1 - 0.9\,\rho(R) & \text{otherwise,} \end{cases} \tag{1}$$

$$\rho(R) = \mathrm{clip}\left(\frac{R - R_{\min}}{P_{90}(H) - R_{\min}},\, 0,\, 1 \right). \tag{2}$$

A return at or above the 90th percentile gives the minimum signal 0.1; the signal grows linearly to 1 at the worst return observed. We call f the *linear ramp*.

1) Trust: The acting specialist's Trust is an exponential average whose rate is the failure signal $D = f(R)$:

$$T_{t+1} = (1 - D)\, T_t + D\, \hat{r}(R), \qquad \hat{r}(R) = \frac{R - P_5(H)}{P_{95}(H) - P_5(H)}, \tag{3}$$

with $\hat{r}$ clipped to $[-2, 5]$, deliberately not to $[0, 1]$, so a specialist that beats the current record still ranks above the rest.

A poor episode moves Trust quickly; a good one only nudges it, which punishes failure fast and rewards success slowly. When $D > 0.1$, the Trust of every non-archived specialist is also blended toward a reference value (the normalized mean of H) by the factor D, which re-opens the competition after a failure.

2) Plasticity: Plasticity $\mathcal{P} \in [0, 1]$ sets the search effort of the episode, and a *controller* sets it from the episode's return (Section III-C3). It selects k of the other $N - 1$ specialists as candidates and evaluates each for at most s steps,

$$\begin{aligned} k &= \min\left(N-1,\, \max\left(2, \lfloor N\mathcal{P} \rfloor\right)\right), \\ s &= s_{\min} + \left(s_{\max} - s_{\min}\right)\mathcal{P}, \end{aligned} \tag{4}$$

in a copy of the current environment (a truncated return is rescaled to a full-episode equivalent). Their Trust is updated with (3), and $\mathcal{P}$ also scales the mutation noise (Section III-D). A confident agent therefore evaluates 2 candidates briefly and mutates gently, while an agent that is failing searches broadly and deeply. Every candidate step is a real environment step and counts against the agent's budget.

3) Plasticity controllers: A Plasticity controller maps the return of an episode to $\mathcal{P}$. Trust always uses the linear ramp (1) as its rate; the Plasticity controller is configured separately. We use five:

- *Linear ramp*, ENA's controller: $\mathcal{P} = f(R)$ (1).
- *Zero*: $\mathcal{P} = 0$ in every episode. This is the minimum search: $k = 2$ candidates for $s_{\min}$ steps each, and mutation noise at its floor. The ablation uses it to switch Plasticity off.
- *Full*: $\mathcal{P} = 1$ in every episode. Every other specialist is evaluated for $s_{\max}$ steps, and mutants receive the largest noise. It serves as the reference for a fixed maximum effort (Section V-B).
- *Quadratic*: $\mathcal{P} = \mathrm{clip}\big(((R_{\max} - R)/(R_{\max} - R_{\min}))^2,\, 0.1,\, 1\big)$, where $R_{\max}$ is the best return seen so far. It stays low for moderate failures and rises steeply only toward the worst return.
- *Fuzzy*: a Mamdani controller [29] with two inputs. The first is the normalized return $\rho(R)$. The second is its recent trend: the mean of the last 5 returns minus the mean of up to 45 returns before them, divided by the same span, and 0 until 20 returns have been seen. Its 9 rules raise Plasticity when the return is low or falling and lower it when the return is high and steady or rising. Inference is min–max with centroid defuzzification.

Like the ramp, the quadratic and fuzzy controllers return 1 until 10 returns have been seen, and stay within $[0.1, 1]$. They are the alternatives compared in Section V-C. At test time $\mathcal{P} = 0$, whatever the controller. When the environment-detection skill confirms a change of physics, H is cleared, so the ramp, quadratic and fuzzy controllers search at full effort until they have seen 10 returns of the new regime.

D. Evolutionary Search

After every training episode, specialists are ranked by Trust, ties going to the older specialist; a specialist's age counts the episodes it has played, and a mutant inherits its parent's. The top 20% form the elite pool. The bottom 80%, excluding archived specialists, are replaced. Of the replaced specialists, 40% become mutants of a random elite, 40% become crossovers of two elites, and the remaining 20% are freshly initialized, which keeps the population diverse. With no archived specialist among them, each generation therefore replaces 32%, 32% and 16% of the population with mutants, crossovers and fresh specialists. A mutant perturbs each weight of its parent with probability p_m by Gaussian noise of standard deviation $0.01 + (\sigma_m - 0.01)\,\mathcal{P}$. A crossover takes each weight from either parent with probability 0.5, after both parents are re-scaled to the current input statistics. New specialists start at the reference Trust.

E. Hall of Fame

The Hall of Fame stores frozen copies of proven specialists, the *gladiators*, at most K per *niche*, a niche being a set of physics recognized by the environment-detection skill (Section III-F). Gladiators are never replaced by the evolutionary step. After each training episode, the acting specialist is considered for induction:

1) *Gate.* Its Trust must stand out from the population, $z = (T - \mu_T)/\max(\sigma_T, 0.1) \geq 1.5\,(1 - 0.5\,\mathcal{P})$, or reach $T \geq 0.5$. High Plasticity lowers the bar while the agent is still searching.
2) *Niche.* The environment-detection skill resolves which niche the episode was played in.
3) *Cloning, not relabelling.* A gladiator that also succeeds in a different niche is cloned into that niche, labelled with the current environment; the original keeps its home niche.
4) *Diversity.* A candidate whose weights have cosine similarity above 0.95 with a gladiator of the same niche only replaces that twin if it has higher Trust.
5) *Capacity.* A candidate fills a free slot of its niche, or replaces the niche's weakest gladiator if it is better.

This design separates what must be kept from what must keep changing.

- *Retention.* A gladiator is a frozen copy, and the Trust stored with it at induction does not decay. It can be displaced only by a candidate of its own niche with strictly higher Trust, so the specialists of one environment can never be evicted by those of another, however long their environment is absent. Gladiators in the population are also exempt from the evolutionary replacement and from the Trust reset that follows a failure.
- *Plasticity.* Within a niche the archive keeps improving: any better candidate takes a free slot or the place of the weakest gladiator, high Plasticity lowers the gate, and success in a new niche creates a clone there instead of moving the original.
- *Olympic re-evaluation.* In domains where returns vary widely with initial conditions (such as continuous-control HalfCheetah), admission based on a single episode risks

retaining specialists that were merely lucky. There ENA uses an *Olympic* Hall of Fame driver: periodically (every 200,000 steps in HalfCheetah), all gladiators of the current niche compete in an "Olympiad" across 3 full episodes on shared seeds. Gladiators that fall below the population reference Trust are evicted, and surviving gladiators are confirmed with their multi-episode Olympic average return. At test time, only Olympic-confirmed gladiators are deployed.

Before testing, gladiators are re-injected into the population round-robin across niches, up to a share of 0.5 of the population.

F. Environment Detection

Following the context-detection idea of da Silva et al. [19], each niche j owns a model of the environment's dynamics, and a niche is recognized by how well its model predicts the transitions the agent actually experiences. Our models are linear-Gaussian:

$$s_{t+1} - s_t = W_j^\top [s_t, a_t, 1] + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}\big(0, \mathrm{diag}(\sigma_j^2)\big)\,, \tag{5}$$

fitted in closed form by ridge least squares from accumulated sufficient statistics, so the detector is gradient-free as well. Conditioning on the action makes the evidence independent of which specialist is playing, and every real transition of every episode is evidence.

1) Detection: The standardized residuals of the current niche's model, with the variance widened by a tolerance τ^2 ($\tau = 3$) to absorb the linear model's own error on nonlinear physics, are monitored with the Predictive-Mixture CuSum (PM-CuSum) of Halme, Poor and Koivunen [5], a recent successor of Page's CUSUM [17]. With $q = \mathcal{N}(0, I)$ the pre-change distribution of the residuals z_t,

$$S_t = \max(S_{t-1}, 0) + \log \frac{\hat{p}_t(z_t)}{q(z_t)}, \tag{6}$$

where $\hat{p}_t$ is a Fixed-Share [30] mixture of Gaussian predictors fitted on sliding windows of $2, 4, \ldots$ recent residuals. An alarm is raised when $S_t > \log \gamma$, which guarantees an average run length to false alarm of at least γ ($\gamma = 10^8$ steps). We depart from the Gaussian example of [5] in one respect: each window also estimates a variance, and that variance is one-sided (never below the pre-change one), because a wrong dynamics model changes the spread of the residuals at least as much as their mean, and can only fit worse, never better.

2) Identification: The recent transitions (up to 500) are recognized as niche j^*, the niche of highest mean per-step log-likelihood, if j^* explains them within a margin of $m = 2$ nats per step of a model fitted on those same transitions. Otherwise the physics are new and a niche is created. Only niches fitted on at least 100 transitions take part.

3) Detect, then verify: An alarm is only a candidate. After 50 post-alarm transitions, the identification test is run against the alarmed niche. If that niche still explains them, the alarm reflected states its model had not yet covered: the transitions are added to the model and the CuSum restarts. Otherwise a change of physics is declared, and the post-alarm transitions seed the new regime's evidence. During the test the dynamics models are never updated: the detector still detects and identifies, but learns nothing new.

4) Late identity binding: A niche is assigned only when a specialist is considered for the Hall of Fame (step 2 above), from the transitions of the episode it just played. The resolved niche's model is refined with them, or a new niche is fitted on them.

5) Chirp detection (HalfCheetah): On HalfCheetah the transition models misfire. Their evidence is the acting specialist's own transitions, and in a contact-rich body every new behavior of a learning population (another gait, a flip, standing still) reaches contact states a niche's model has not seen, which read as new physics. On the detection benchmark of Section V-E, averaged across 30 confirmation seeds, they confirm 26.3 changes and create 21.8 niches on average for three environments (where each individual run produces integer counts), and even with a single fixed gait played throughout they still confirm 16.2 changes and create 6.5 niches on average: the agent's own transitions are poor evidence of the physics.

ENA therefore plays a chirp, as in system identification with a known input signal [31]. At every episode end it resets the environment and plays the same 64-step action sequence, whatever any specialist would do, and compares the observations it produces across niches. The excitation is multiresolution, a wavelet train: the Mexican hat wavelet played at dyadic scales, from a segment of half the chirp down to the finest, so that slow and fast responses of the body are both excited. The response, the observations minus the chirp's starting observation, is decomposed per observation channel into a Haar wavelet packet tree of depth 3 [32], dyadic and non-overlapping like the discrete wavelet transform [33] but splitting every band, which gives 8 frequency bands of equal width; one log-energy is kept per band. Wavelet features of a system's response to a known excitation are the standard signature of a change of physics in structural damage detection and machine fault diagnosis [34], [35]. Probing an environment to infer its dynamics has been learned in reinforcement learning [36]; ENA's chirp is fixed, so its response cannot depend on the agent. The transforms are computed with PyWavelets [37].

Each niche j keeps the mean μ_j of its n_j chirps' feature vectors; the per-feature variance σ^2 is pooled over niches, as it comes from the environment's reset noise. The mean $\bar{f}$ of the last $w = 4$ chirps is tested against the current niche with

$$T_j = \left(\frac{1}{d} \sum_{k=1}^{d} \frac{(\bar{f}_k - \mu_{j,k})^2}{\sigma_k^2 \,(1/w + 1/n_j)} \right)^{1/2}, \tag{7}$$

which is about 1 under the same physics (d features). Above the margin 3 a change is confirmed, and the most recent chirps identify the new physics: the known niche of lowest T_j within the margin, otherwise a new niche (in training only). A niche

is judged once it holds 8 chirps. At test, each chirp's Gaussian log-likelihood under each niche is the dynamics evidence L_j of the selection (8). The chirp costs 64 steps per episode (6.4 % of a HalfCheetah episode), counted in ENA's step budget, and the environment's random stream is restored after it, so that episode starts are unchanged. CartPole keeps the transition models, which detect its changes (Appendix A-C).

G. Specialist Selection

At test time the weights are frozen and $\mathcal{P} = 0$; the only adaptation left is the choice of which gladiator acts. We use Bayesian Policy Reuse (BPR) [6], with the niches as types and every gladiator as a policy. The agent keeps a belief β over niches and plays the gladiator of highest expected utility $\sum_j \beta(j)\,\mu_{j\pi}$, where $u = R/R_{\text{solved}}$ is the return relative to the one at which the environment counts as solved (so solving reads as $u = 1$), and $\mu_{j\pi}$ is the mean utility of policy π in niche j. After each episode,

$$\beta'(j) \propto \beta(j)\,\mathcal{N}\big(u;\,\mu_{j\pi}, 0.2^2\big)\;e^{L_j}, \tag{8}$$

where L_j is the log-likelihood of the episode's transitions under niche j's dynamics model (5): the dynamics evidence and the return evidence are combined in one posterior. Two changes adapt BPR to our setting.

- BPR builds its performance model offline, by playing every policy in every type. Ours is learned online: a prior of utility 1 for a gladiator in its own niche and 0.5 elsewhere, worth 1 observation, updated with every observed utility weighted by the posterior.
- BPR assumes one fixed type per trial. To follow a changing environment, a Fixed-Share step [30] mixes back a uniform share: $\beta \leftarrow 0.95\,\beta' + 0.05/|\mathcal{J}|$. A confirmed change of physics also resets the belief to uniform.

IV. Experimental Setup

We evaluate ENA in two non-stationary domains built on Gymnasium [38]: the discrete-action `CartPole-v1` [39] and the continuous-control MuJoCo [40] `HalfCheetah-v5`. In both, the agent trains on three environments in sequence, without being told when the environment changes, and is then tested, frozen, on five environments, two of which it has never seen.

A. Environments

1) CartPole: Each environment changes the gravity g, the pole mass m_p and the pole half-length ℓ (Table I). The training sequence is Mars, Jupiter, Earth. The test sequence interleaves them with two unseen environments: Mars, Neptune, Jupiter, Brown-dwarf, Earth. Neptune is close to Earth; Brown-dwarf extrapolates beyond Jupiter's gravity. An episode ends when the pole falls or after 500 steps; a return of 475 or more counts as a success, the threshold at which Gymnasium considers `CartPole-v1` solved.

TABLE I
CartPole Environments

Environment	g (m/s^2)	m_p (kg)	ℓ (m)	Seen in training
Mars-Light	3.7	0.05	0.1	yes
Jupiter-Heavy	24.8	0.5	1.5	yes
Earth-Standard	9.8	0.1	0.5	yes
Neptune	11.15	0.1	0.5	no
Brown-dwarf	50	0.1	0.5	no

2) HalfCheetah: Each environment changes the gravity, the ground friction (applied to every contact surface, since MuJoCo uses the larger friction of two touching bodies) and the wind (Table II). The training sequence is Earth, with a soft head wind, Heavy-Rough (1.5 times Earth's gravity, on rough ground) and Mars-Slippery (Mars's gravity on a slippery floor); the test sequence adds the unseen Ice-Floor and High-Wind, which keep Earth's gravity and change only the ground or the strength of the wind. Gymnasium's registered reward threshold for `HalfCheetah-v5` is 4,800, shown as a reference line in the figures; no agent reaches it reliably on the non-stationary stream, so *retention* (Section IV-D) replaces reliability as the primary metric.

TABLE II
HalfCheetah Environments

Environment	g (m/s^2)	Friction	Wind (m/s)	Seen
Earth-Standard	9.8	0.4	-5	yes
Heavy-Rough	14.7	1	0	yes
Mars-Slippery	3.7	0.2	0	yes
Ice-Floor	9.8	0.05	0	no
High-Wind	9.8	0.4	-20	no

ENA detects HalfCheetah's environments with its chirp (Section III-F5) instead of the transition models.

B. Protocols

1) Skill ablation (CartPole): Each ENA variant trains for 500 episodes per environment and is then tested for 200 episodes per environment. We run 30 independent replicates. Replicate i seeds Python's, NumPy's and TensorFlow's random number generators with $1000 + i$ before each agent is built, trained and tested, and seeds every environment from the same number, so all the agents of a replicate start from the same random state and meet the same initial states: they differ only in what they are.

2) Comparison with gradient-based agents: Comparing a population method with single-policy agents is only fair at equal interaction. Every agent therefore trains on each environment for the same budget of environment steps: 2,500,000 in CartPole and 1,000,000 in HalfCheetah. ENA's budget includes every step it spends evaluating candidates (4), holding Olympiads, and playing its chirp excitation (III-F5), not only the learning episodes. Because ENA spends 84.9% of its CartPole training steps evaluating candidates, it plays far fewer episodes within the same budget: about 815 per environment on CartPole, against 6,759 for DQN, 5,791 for PPO and 5,927

for SAC-Discrete. On HalfCheetah, ENA plays about 214 episodes per environment, against 1,000 for PPO and 1,000 for SAC. Training is therefore compared against environment steps, not episodes, so every agent spans the same budget. The frozen test needs no such normalization, since every agent plays the same number of test episodes. No agent observes the physical parameters or is told when the environment changes; Appendix A lists what each agent receives. On CartPole, every 5,000 training steps the agent is evaluated, frozen, for 5 episodes on a separate copy of the current environment; these checks never stop or alter training and their steps are not counted as training; they record the *first solve*, the number of training steps into an environment at which the evaluation mean first reaches the success threshold. HalfCheetah uses no intermediate evaluation checks: no agent clears its threshold reliably on the non-stationary stream, so first-solve is not measured there. After training, all agents are tested frozen on 50 episodes per environment, exactly as in the ablation. The CartPole comparison runs 50 replicates; its 3 comparisons (ENA-Combined against each baseline on test reliability) form one Holm family. The HalfCheetah comparison runs 20 replicates; its two comparisons (ENA-Combined against PPO and SAC on left-behind retention) form one Holm family.

C. Agents

1) ENA variants: The three skills under study are switched on and off as in Table III; every other setting is shared. "Plasticity off" means the zero controller (Section III-C3): $\mathcal{P} = 0$, the minimum search. "Plasticity on" means the linear ramp. Environment detection only acts through the Hall of Fame, whose archive it organizes into niches, so it is never tested without it. A random agent gives the floor. The full configuration is listed in Appendix B.

TABLE III
ENA VARIANTS IN THE SKILL ABLATION

Variant	Plasticity	HoF	Detection
ENA-NoSkill	–	–	–
ENA-Plasticity	✓	–	–
ENA-HoF	–	✓	–
ENA-HoF+EnvDetection	–	✓	✓
ENA-Plasticity+HoF	✓	✓	–
ENA-Combined	✓	✓	✓

2) Gradient-based baselines: On CartPole we compare ENA-Combined with DQN [41], PPO [42] and the discrete-action variant of SAC [43]; on HalfCheetah with PPO and SAC [44]. All are our own implementations, with common hyperparameters for these benchmarks (Appendix B) and two changes that the non-stationary stream made necessary, both found in full-length runs before the reported study:

- *PPO uses a constant learning rate.* With the usual linear annealing to zero over the training budget, PPO reaches its last environment with almost no learning rate left; a learner in a non-stationary stream has no reason to learn less in a later environment than in an earlier one.
- *SAC-Discrete uses a target entropy of* $0.5 \log |\mathcal{A}|$ instead of $0.98 \log |\mathcal{A}|$. With CartPole's two actions, $0.98 \log 2$ is nearly the maximum entropy; the policy cannot reach it, and the temperature grows without bound, collapsing the agent to random play. Over 148k steps of the first training environment (median of 3 seeds), the temperature rose from 1.0 to 21,048, and the critics' values to 125,066. A ratio of 0.89 diverged in the same way (temperature 4,907, values 40,174). At 0.5 the temperature peaked at 3.7 and the values at 121.

During the frozen test, the baselines act greedily, and ENA plays with its weights frozen and Plasticity at zero, adapting only through its choice of gladiator.

D. Metrics

- *Reliability* (CartPole): the percentage of test episodes whose return reaches the success threshold, over all five test environments.
- *Retention* (HalfCheetah): the test return as a percentage of the end-of-phase training return (the mean return of the episodes ending in the last 1,000,000 / 10 steps of a phase), for each training environment; and the mean of that percentage over the environments left behind (all training environments except the last).
- *Mean return* per environment.
- *Stability gap*: the mean return of the 20 episodes before the first environment switch minus that of the 20 episodes after it; values near zero mean the switch costs nothing.
- *Steps to first solve* per training environment (CartPole comparison only), at the 5,000-step resolution of the evaluation checks.
- *Compute*: training environment steps, ENA's candidate-evaluation steps among them, and wall-clock training time.

Every metric is computed per replicate and reported as mean and standard deviation over replicates. Each difference between two agents is reported in three ways: the difference of means with its 95% confidence interval, the effect size d (the difference over the average of the two standard deviations), and a Welch's t-test at $\alpha = 0.05$.

The skill ablation makes 7 planned comparisons: each skill added on its own and on top of the others. We treat them as one family and correct their p-values with Holm's method. The number of replicates was fixed before the runs by a power analysis. With 30 replicates per variant, a two-sided test at the family's most stringent level ($0.05/7$) has 80% power for effects of $d \geq 0.94$.

V. RESULTS AND PERFORMANCE ANALYSIS

All numbers below are means over independent replicates, with the standard deviation in parentheses where relevant; p is the two-sided Welch's t-test.

A. Skill Ablation

Table IV and Fig. 2 report the frozen test over 30 replicates. With all three skills, **ENA-Combined** reaches a test reliability

of 83.2% (16.1), against 38.5% (20.0) without any skill, and is better than every other variant. Its smallest margin, over ENA-Plasticity, is +19.9 points (95% CI 11.9 to 27.9; $d = 1.29$; $p < 0.001$).

Adding the skills one at a time shows how they depend on each other. Differences are in reliability points, with their 95% interval and Holm-corrected p-value:

- *Plasticity is the largest single contribution*: +24.8 over ENA-NoSkill (15.7 to 33.9; $p < 0.001$), and +35.0 on top of the other two skills (22.4 to 47.6; $p < 0.001$). Searching in proportion to failure is what produces good specialists in the first place.
- *An archive alone does not help*: the Hall of Fame changes ENA-NoSkill's reliability by -13.8 (-24.6 to -2.9; $p = 0.027$) and ENA-Plasticity's by -4.4 (-14.9 to 6.2; $p = 0.408$). Without knowing which physics a specialist was good for, the agent cannot tell which one to recall.
- *Environment detection turns the archive into a memory*: organized by niche, the archive gains +23.5 without Plasticity (9.8 to 37.2; $p = 0.003$), and +24.3 with it (13.5 to 35.1; $p < 0.001$). Detection pays off once Plasticity has produced specialists worth remembering.

The full combination also removes most of the dependence on the initial population: the standard deviation of reliability across replicates drops from 21.9–30.3 points for the archive-only variants to 16.1. On the two environments never seen in training, ENA-Combined's mean return is 490.4 on Neptune, whose physics its detector recognizes as Earth's (Appendix A). On Brown-dwarf it is 367.2 (153.9). Brown-dwarf's gravity is twice Jupiter's, and its physics are recognized as no known niche, so ENA-Combined does not solve it reliably.

Statistical power. Every effect the conclusions rest on is resolved. Plasticity, environment detection and their combination each have d between 0.89 and 1.47, every one of their intervals excludes zero, and ENA-Combined's margin over each variant does too.

The two Hall of Fame effects without detection are not resolved in size. On its own, the archive lowers reliability: the interval excludes zero, but measuring the size with 80% power would take 60 replicates. On top of Plasticity, the effect ($d = -0.22$) would take 539.

We did not add replicates for them, for two reasons:

- Neither is a claim of this paper. The Hall of Fame is proposed together with environment detection, and that effect is resolved.
- The interval already answers the design question. Without detection, the archive does not raise ENA-Plasticity's reliability by more than 6.2 points.

B. Compute Cost of Plasticity

Plasticity is also the expensive skill: variants with it train in 34.9–38.9 minutes per replicate, against 6.0–7.5 without it, because candidate evaluation dominates the cost. Scaling the search to the failure signal with the linear ramp, instead of always searching at full effort, is what keeps that cost bounded.

TABLE IV
SKILL ABLATION, FROZEN TEST: MEAN RETURN PER ENVIRONMENT AND RELIABILITY (30 REPLICATES)

Agent	Mars	Neptune	Jupiter	Brown	Earth	Rel. (%)
Random	9.0	21.8	33.7	16.3	22.2	0.0 ± 0.0
ENA-NoSkill	311.8	321.7	144.8	170.1	321.4	38.5 ± 20.0
ENA-Plasticity	279.8	416.3	350.7	313.1	419.4	63.3 ± 14.8
ENA-HoF	326.6	252.4	128.3	108.3	299.9	24.7 ± 21.9
ENA-HoF+EnvDetection	404.5	393.7	272.5	208.5	414.8	48.3 ± 30.3
ENA-Plasticity+HoF	474.8	352.1	277.9	281.5	450.7	58.9 ± 24.6
ENA-Combined	475.5	490.4	464.9	367.2	490.5	83.2 ± 16.1

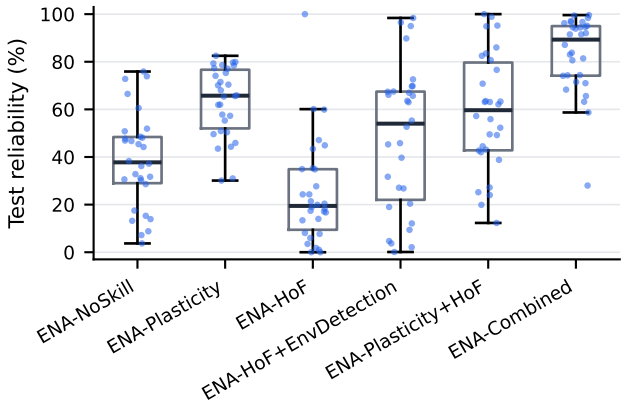


Fig. 2. Test reliability of each ENA variant; each dot is one of the 30 replicates.

We ran ENA-Combined once with the full controller ($\mathcal{P} = 1$, Section III-C3), and compared it with the same seeded replicate under the linear ramp (ENA-Combined) and under the zero controller (ENA-HoF+EnvDetection). The three runs spent 57,890,361, 4,617,335 and 27,816 environment steps evaluating candidates during training. Full search therefore costs 13 times the linear ramp, which in turn costs 166 times the minimum.

One replicate is enough for this measurement. At full effort every generation evaluates every other specialist for up to $s_{\max}$ steps, so its cost is set by construction and varies only where a candidate's episode ends early. The linear ramp's cost does vary from run to run, with a standard deviation of 596,722 steps over the ablation's 30 replicates.

Test reliability in this replicate was 90.3% at full effort, 74.1% with the ramp, and 55.3% without search. Most of the gap between full effort and the ramp comes from Brown-dwarf, the extrapolation environment: a mean return of 410.0 against 61.5.

A single run supports no test of reliability, so we place it among the ramp's own replicates instead. 15 of the ramp's 30 ablation replicates reached at least the reliability of full search, and this replicate is one of the ramp's weaker runs. Full search's result is therefore one the ramp reaches in about half its runs, with 13 times less candidate evaluation.

C. Shape of the Plasticity Controller

ENA's controller, the linear ramp (1), is one of several ways to turn a return into a search effort. We compared it with the two alternatives of Section III-C3, the quadratic and the fuzzy controller, each replacing only ENA-Combined's Plasticity controller.

Each alternative ran for 30 replicates with the ablation's seeds. Every agent of a replicate starts from the same random state, so the ablation's ENA-Combined is the linear-ramp run of the same replicate. It serves as the reference without being re-run.

The decision rule was fixed before the runs. An alternative replaces the ramp if its test reliability is significantly higher, or not significantly lower with significantly fewer candidate-evaluation steps. Both comparisons with the ramp form one family (Holm). At 30 replicates, this family has 80% power for $d \geq 0.81$.

The results, with the 95% interval and the Holm-corrected p-value of each difference:

- *Linear ramp (reference):* test reliability 83.2% (16.1), with 4,226,921 candidate steps.
- *Fuzzy:* 83.9% (11.4).
  - Reliability: +0.7 points (-6.5 to 7.9; $p = 0.854$).
  - Cost: +481,852 candidate steps (187,089 to 776,616; $p = 0.004$), for 4,708,773 in total.

  The fuzzy controller searches more and gains nothing measurable. Its interval rules out a gain of more than 7.9 points.
- *Quadratic:* 89.3% (7.7).
  - Reliability: +6.0 points (-0.5 to 12.6; $p = 0.141$).
  - Cost: -294,127 candidate steps (-633,393 to 45,140; $p = 0.088$).

  The quadratic controller was more reliable and cheaper in this sample, but neither difference is significant. The observed effect ($d = 0.48$) is below the detectable one, and resolving it with 80% power would take about 84 replicates per controller.

By the rule, the controller kept is the linear ramp. We do not claim that the ramp is better than the quadratic controller: the data leave that open, and the interval of their difference includes both no difference and a gain of over ten points for the quadratic controller.

As an exploratory observation, not part of the planned comparison, the quadratic controller's reliability varied less across replicates (standard deviation 7.7 against 16.1). Settling the question would need a separate study, planned for that effect size on fresh seeds.

D. Comparison with Gradient-Based Agents on CartPole

Tables V and VI and Figs. 4 and 3 report 50 replicates under the equal budget of 2,500,000 environment steps per environment.

1) Performance and forgetting: **ENA-Combined** reaches a test reliability of 83.9% (11.7), against 74.5% for SAC-Discrete, 73.8% for PPO and 50.6% for DQN (Fig. 3). Each of the three differences is significant after Holm correction over the 3 comparisons:

- *SAC-Discrete:* ENA leads by +9.4 points (95% CI 5.0 to 13.7; $d = 0.85$; Holm $p < 0.001$).
- *PPO:* ENA leads by +10.1 points (95% CI 5.8 to 14.3; $d = 0.94$; Holm $p < 0.001$).
- *DQN:* ENA leads by +33.3 points (95% CI 25.1 to 41.4; $d = 1.63$; Holm $p < 0.001$).

The difference comes from one environment. The gradient-based agents end training on Earth. On Jupiter, the environment they trained on before Earth, their mean return falls to 21.6 (PPO), 52.1 (SAC-Discrete) and 75.2 (DQN): they have forgotten it. ENA keeps 461.3, because its Jupiter specialists are archived and recalled when Jupiter returns.

Elsewhere ENA does not lead (Table V). SAC-Discrete scores slightly higher on the other environments it trained on, and on Neptune, which it never saw: on Mars 495.7 against 480.2 ($p = 0.026$), on Earth 500.0 against 498.1 ($p = 0.049$), and on Neptune 500.0 against 492.9 ($p < 0.001$). On Brown-dwarf, the other unseen environment, ENA is on par with PPO and SAC-Discrete: 381.6 against 379.5 and 393.5 ($p = 0.944$ and $p = 0.695$). These per-environment comparisons are uncorrected and descriptive. ENA trades a few points on the environments a gradient-based agent still remembers for keeping the one it forgets.

2) Adaptability: ENA does not reach the success threshold first. PPO first solves each training environment in 8,835, 56,407 and 5,100 steps, ENA in 104,348, 206,892 and 45,343 ($p < 0.001$, $p = 0.009$ and $p < 0.001$). Against the other two baselines the picture is mixed (Table VI):

- *Mars:* ENA solves it sooner than DQN (234,009 steps; $p < 0.001$) and as soon as SAC-Discrete (109,515; $p = 0.861$).
- *Jupiter:* ENA is level with both (141,966 and 163,221; $p = 0.252$ and $p = 0.435$).
- *Earth:* ENA is later than both (21,068 and 6,856; $p < 0.001$ and $p < 0.001$).

ENA's search spends steps on many candidates before one is good enough. What it gains in exchange is that, once found, a solution is kept.

3) Compute: Under the same number of environment steps, ENA trained in 65.3 minutes of wall-clock time per replicate, against 97.1 (DQN), 119.0 (SAC-Discrete) and 140.3 (PPO), on the same CPU and with implementations in the same framework. ENA needs no backpropagation: its cost is forward passes and environment steps, 84.9% of which go to evaluating candidates.

E. Environment Detection on HalfCheetah

The detector's inputs are the agent's own, so detection can be evaluated without training. The detection benchmark plays scripted episodes in Earth, Heavy-Rough and Mars-Slippery (40 each), changing the acting policy every 4 episodes

TABLE V
CARTPOLE COMPARISON, FROZEN TEST (50 REPLICATES)

Agent	Mars	Neptune	Jupiter	Brown	Earth	Rel. (%)
DQN	401.6	443.7	75.2	238.2	442.0	50.6 ± 26.4
PPO	488.0	497.9	21.6	379.5	498.0	73.8 ± 9.6
SAC-Discrete	495.7	500.0	52.1	393.5	500.0	74.5 ± 10.2
ENA-Combined	480.2	492.9	461.3	381.6	498.1	83.9 ± 11.7

TABLE VI
CARTPOLE COMPARISON: TRAINING STEPS TO FIRST SOLVE PER ENVIRONMENT (50 REPLICATES)

Agent	Mars	Jupiter	Earth
DQN	234,009 ± 100,298	141,966 ± 59,693	21,068 ± 25,218
PPO	8,835 ± 2,398	56,407 ± 27,248	5,100 ± 707
SAC-Discrete	109,515 ± 78,132	163,221 ± 23,671	6,856 ± 8,734
ENA-Combined	104,348 ± 192,453	206,892 ± 391,316	45,343 ± 23,681

among near-still, uniformly random and sinusoidal gaits, the same family in every environment so that only the physics differ, and feeds the detector exactly the calls ENA makes, with a niche assignment after every episode. A run meets the target when it confirms both switches within 5 episodes, triggers no false alarms inside an environment, and gives each environment its own niche covering at least 90 % of its episodes. To obtain statistically reliable estimates, each detector is evaluated across 30 independent benchmark runs (seeds), each drawing different policies and resets. Intervals are exact (Clopper–Pearson) at 95 %; comparisons are exact McNemar tests on seed-paired outcomes, Holm-adjusted as one family. As a sanity check of the benchmark criteria, CartPole's transition-model detector (which detects CartPole's changes in the studies above) meets the target on 26 of 30 runs on the corresponding CartPole benchmark (catching every switch without false alarms in 86.7% of runs).

On HalfCheetah, however, the transition-model detector, with settings calibrated for HalfCheetah, meets the target on

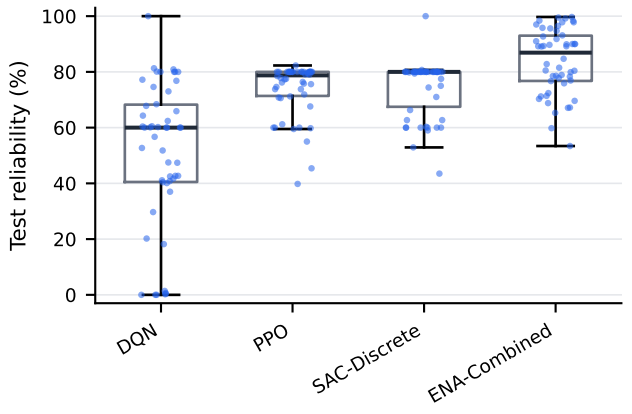


Fig. 3. Test reliability on CartPole across 50 replicates under equal environment step budgets; each dot is one replicate.

0 of 30 runs: it confirms an average of 26.3 changes (24.5 of them inside an environment) and creates an average of 21.8 niches, and only 21.9 % of the worst environment's episodes fall in its niche. With one fixed gait in every environment the same detector still confirms 16.2 changes and creates 6.5 niches on average, and merges two environments into one niche on 14 of the seeds: changing behavior is one cause of its errors, and the agent's own transitions do not separate these physics even when the behavior is fixed.

Table VII ablates the chirp's excitation and response features across 30 benchmark seeds. The excitation matters most: the Mexican-hat wavelet train meets the target on at least 29 of the 30 seeds with every response feature, and on every seed with 3 of them. No linear chirp, sine or impulse meets it on any seed, and the best configuration without a wavelet train, an Exponential chirp with SWT Haar features, on 4 seeds. The wavelet's shape matters too: the Haar and db4 [45] trains depend strongly on the features, the db4 train with a Haar DWT meeting the target on only 12 seeds. ENA uses the Mexican-hat train with Haar wavelet packets (bold), which meets it on 30 of the 30 seeds.

On the confirmation runs, this chirp meets the target on all 29 of 30 seeds (95 % interval 82.8–99.9 %): it confirms a false change on 1 of them, catches every switch with a delay of at most 0 episodes, and keeps at least 75 % of every environment's episodes in its niche. It meets the target more often than the transition-model detector ($p < 0.001$), and on the ablation seeds more often than the best configuration without a wavelet train ($p < 0.001$).

TABLE VII
CHIRP DETECTOR ABLATION ON THE HALFCHEETAH DETECTION BENCHMARK: BENCHMARK SEEDS (OF 30) ON WHICH AN EXCITATION AND RESPONSE FEATURES CATCH BOTH SWITCHES, CONFIRM NO FALSE CHANGE AND GIVE EACH ENVIRONMENT ITS OWN NICHE

Excitation	Raw	DWT Haar	DWT db4	SWT Haar	Packets Haar
Linear chirp	0/30	0/30	0/30	0/30	0/30
Exponential chirp	0/30	0/30	0/30	4/30	1/30
Sine	0/30	0/30	0/30	0/30	0/30
Impulse	0/30	0/30	0/30	0/30	0/30
Wavelet train (Haar)	26/30	12/30	28/30	2/30	14/30
Wavelet train (db4)	0/30	12/30	17/30	13/30	26/30
Wavelet train (Mexican hat)	30/30	29/30	30/30	29/30	**30/30**

F. Continuous Control: HalfCheetah

Tables VIII and IX and Fig. 5 report 20 replicates. No agent reaches the 4,800 threshold reliably on the non-stationary stream (ENA 0.0%, PPO 0.9%, SAC 47.1%), so *retention* is the primary metric. Episode-by-episode test curves and complete training trajectories across all environments are documented in Appendix D (Figs. 6 and 7).

1) Performance and forgetting: The left-behind environments are Earth-Standard, trained first, and Heavy-Rough, trained second; Mars-Slippery, the last training environment, is not left behind. **ENA-Combined** retains 115.6% (16.6) of its left-behind performance at test: its test returns (2000.1 on Earth-Standard, 2417.1 on Heavy-Rough) exceed its end-of-phase training returns (1,805 and 2,133). This retention above

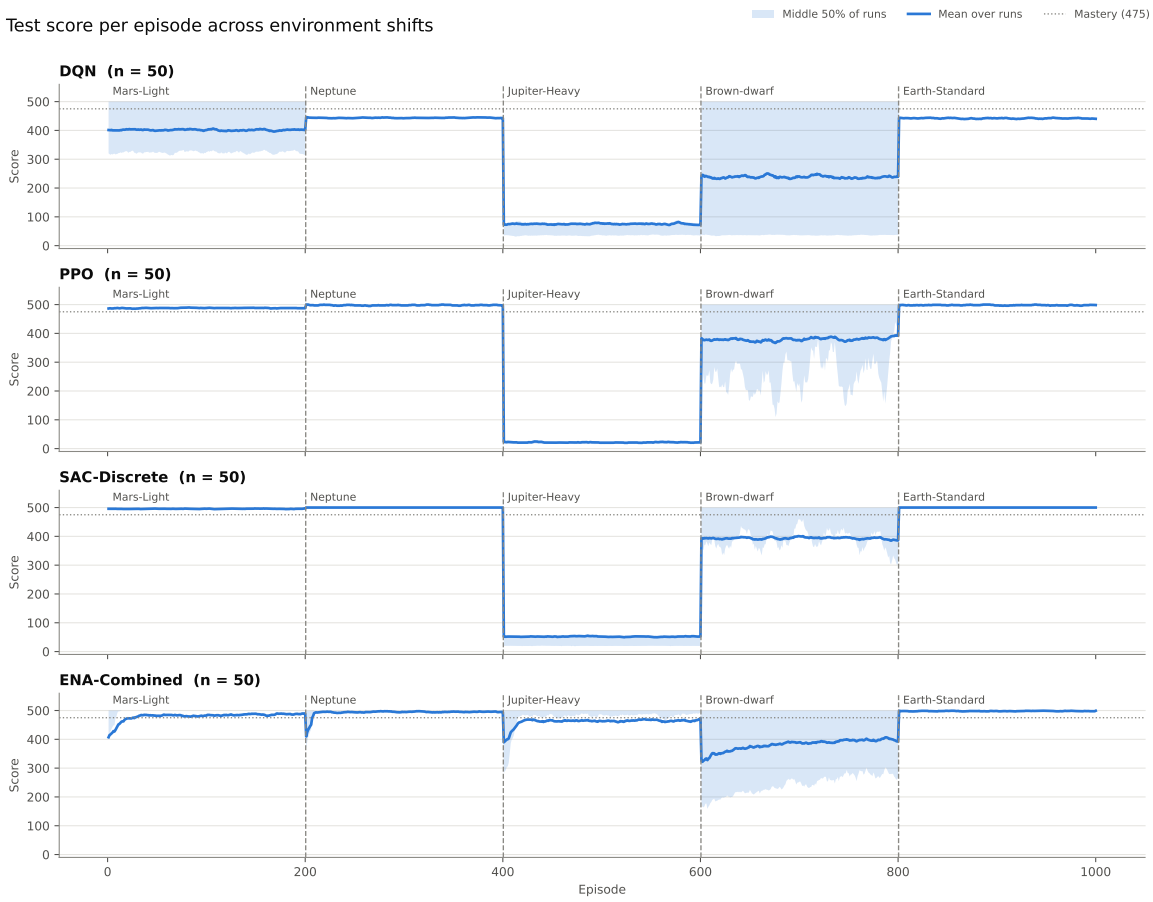


Fig. 4. Frozen test on CartPole: mean return per episode across 50 replicates (line) and the middle 50% of replicates (band). Dashed lines mark environment switches; Neptune and Brown-dwarf were never seen in training.

100% occurs because end-of-phase training averages across active evolutionary search (including exploratory mutations and candidate evaluations), whereas test evaluation deploys the top-performing specialist recalled from the Hall of Fame frozen and without exploration noise. In contrast, PPO retains only 39.2% (17.9) and SAC 34.4% (13.1). Both differences are significant after Holm correction over the two comparisons:

- *PPO:* ENA leads by +76.3 percentage points (95% CI 65.3 to 87.4; $d = 4.42$; Holm $p < 0.001$). PPO ends Heavy-Rough at 4,793 but tests at 1301.8 (28.3% retained): it has forgotten it.
- *SAC:* ENA leads by +81.2 percentage points (95% CI 71.6 to 90.8; $d = 5.44$; Holm $p < 0.001$). SAC ends Heavy-Rough at 7,544 but tests at 1301.1 (18.0%).

On absolute test return ENA is not the strongest agent. SAC trains to a high absolute return in each environment and its scores on the last training environment and the unseen ones substantially exceed ENA's: on Mars-Slippery 10146.1 against 1772.8 ($p < 0.001$), on Ice-Floor 6773.9 against 1915.3 ($p < 0.001$). On Heavy-Rough, the environment both baselines forgot, ENA leads SAC by +1116.0 and PPO by +1115.4 (both $p < 0.001$). These per-environment comparisons are uncorrected and descriptive (Table IX).

2) Compute: ENA trained in 32.0 minutes per replicate (1.7), against 115.3 (PPO) and 281.1 (SAC), on the same hardware. Of ENA's 1,000,000 training steps per environment, 2,376,493 total (across all three environments) went to candidate evaluation, leaving about 214 episodes per environment for learning — versus 1,000 for PPO and 1,000 for SAC.

TABLE VIII
HALFCHEETAH COMPARISON: END-OF-PHASE AND TEST RETURN, AND RETENTION, OF THE ENVIRONMENTS LEFT BEHIND, AND WALL-CLOCK TRAINING TIME (20 REPLICATES)

Agent	Earth end	Earth test	Earth ret. (%)	Heavy end	Heavy test	Heavy ret. (%)	Retention (%)	Time (min)
PPO	4,543	2,208	50.1	4,793	1,302	28.3	39.2	115.3
SAC	8,635	4,353	50.8	7,544	1,301	18.0	34.4	281.1
ENA-Combined	1,805	2,000	116.3	2,133	2,417	114.9	115.6	32.0

TABLE IX
HALFCHEETAH COMPARISON, FROZEN TEST: MEAN RETURN PER ENVIRONMENT (20 REPLICATES)

Agent	Earth	Ice	Heavy	High	Mars
PPO	2208.3 ± 1013.3	2202.7 ± 1000.1	1301.8 ± 1086.9	929.2 ± 908.2	2506.3 ± 1186.6
SAC	4352.5 ± 980.7	6773.9 ± 2214.0	1301.1 ± 1217.9	2642.1 ± 895.9	10146.1 ± 974.7
ENA-Combined	2000.1 ± 491.6	1915.3 ± 502.7	2417.1 ± 538.6	995.7 ± 511.4	1772.8 ± 374.7

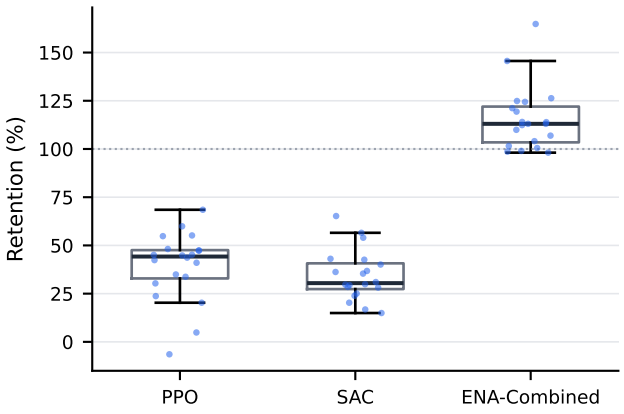


Fig. 5. Left-behind retention on HalfCheetah (20 replicates); each dot is one replicate. ENA retains its left-behind specialists at test; PPO and SAC forget them.

G. Limitations

- ENA's transition models are linear. They separate the CartPole environments, but physics a linear model cannot tell apart (such as Earth and Neptune) share a niche. On HalfCheetah they fail, because each new behavior reaches unseen contact states; there ENA plays a chirp instead (Section III-F5), at a fixed cost per episode, a design chosen and evaluated on a scripted benchmark of the three training environments.
- ENA solves a new environment later than PPO, the price of a gradient-free search.
- The compute comparison is in wall-clock time on one machine, with our own implementations of every agent.
- The compute-cost study of Plasticity is a single replicate per setting.
- ENA's absolute returns on HalfCheetah are well below SAC's on every environment it adapts to. ENA plays only 214 learning episodes per environment — versus 1,000 for SAC — because the fixed step budget is shared between learning episodes, the chirp excitation steps, and candidate evaluation. A larger per-environment step budget would reduce this asymmetry.

VI. Conclusion and Future Work

We introduced the Evolutionary Neuroplastic Agent (ENA), a gradient-free agent for non-stationary environments that keeps a memory of specialists organized by the physics it detects. On non-stationary CartPole, its skills proved complementary: searching in proportion to failure produces good specialists, and detecting and recognizing environments turns the archive of those specialists into a memory the agent can use. Together they raise test reliability from 38.5% to 83.2% over 30 replicates. At an equal budget of environment steps, ENA retained an environment that DQN, PPO and SAC-Discrete forgot, and reached a higher test reliability in less wall-clock time. It was, however, slower than PPO to solve each new environment for the first time, and SAC-Discrete scored slightly higher on the environments it did not forget: ENA trades speed of first adaptation and a few points of peak return for retention. On continuous-control HalfCheetah the same trade-off appears in a sharper form: ENA retains 115.6% of its performance on the environments it trained on first, against 39.2% for PPO and 34.4% for SAC; SAC, however, reaches substantially higher absolute returns within each environment, because ENA can play only 214 learning episodes per environment on the same step budget.

Several directions follow. The linear dynamics models separate our environments, but physics a linear model cannot tell apart share a niche; nonlinear models, still fitted without gradients, would sharpen identification in richer domains. ENA's search operators are deliberately simple, and the evolution strategies that Nisioti et al. [2] found to balance adaptation and forgetting well could replace them, with environment detection and the Hall of Fame on top. Specialist selection could also be learned: a contextual bandit that maps the detector's evidence to the specialist to deploy, instead of the fixed performance model of Bayesian policy reuse. A promising hybrid direction is combining gradient-free neuroevolution with gradient-based learners: gradient updates (such as SAC or PPO) could act as fast intra-niche policy optimizers to accelerate initial adaptation and reach higher peak returns within an environment, while ENA's population dynamics, Plasticity, and physics-indexed Hall of Fame protect against catastrophic forgetting across non-stationary regime switches. Finally, the population's evaluations are independent and parallelize naturally on accelerators, which would allow larger populations and longer non-stationary streams.

Code Availability

The full implementation of the Evolutionary Neuroplastic Agent, the experiment runners, the baselines, the LaTeX source of this paper and the scripts that generate its tables and figures are publicly available:

- **GitHub Repository:**
  https://github.com/blasvicco/ena

References

[1] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in *Psychology of Learning and Motivation*. Academic Press, 1989, vol. 24, pp. 109–165.

[2] E. Nisioti, A. Cossu, K. Korte, and S. Risi, "Continual reinforcement learning with neuroevolution," *arXiv preprint arXiv:2610.01583*, 2026.

[3] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," *arXiv preprint arXiv:1504.04909*, 2015.

[4] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, "Robots that can adapt like animals," *Nature*, vol. 521, no. 7553, pp. 503–507, 2015.

[5] T. Halme, H. V. Poor, and V. Koivunen, "Adaptive sequential change detection using mixtures of predictive distributions," *arXiv preprint arXiv:2606.05072*, 2026.

[6] B. Rosman, M. Hawasly, and S. Ramamoorthy, "Bayesian policy reuse," *Machine Learning*, vol. 104, no. 1, pp. 99–127, 2016.

[7] K. Khetarpal, M. Riemer, I. Rish, and D. Precup, "Towards continual reinforcement learning: A review and perspectives," *Journal of Artificial Intelligence Research*, vol. 75, pp. 1401–1476, 2022.

[8] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[9] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, "Experience replay for continual learning," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[10] S. Kessler, J. Parker-Holder, P. Ball, S. Zohren, and S. J. Roberts, "Same state, different task: Continual reinforcement learning without interference," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 7, 2022, pp. 7143–7151.

[11] K. O. Stanley, J. Clune, J. Lehman, and R. Miikkulainen, "Designing neural networks through neuroevolution," *Nature Machine Intelligence*, vol. 1, no. 1, pp. 24–35, 2019.

[12] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, "Evolution strategies as a scalable alternative to reinforcement learning," *arXiv preprint arXiv:1703.03864*, 2017.

[13] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune, "Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning," *arXiv preprint arXiv:1712.06567*, 2017.

[14] S. Khadka and K. Tumer, "Evolution-guided policy gradient in reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 31, 2018, pp. 1188–1200.

[15] Y. Jin and J. Branke, "Evolutionary optimization in uncertain environments—a survey," *IEEE Transactions on Evolutionary Computation*, vol. 9, no. 3, pp. 303–317, 2005.

[16] L. Lillo and N. Cheney, "Beyond single-model optimization: Preserving plasticity in continual reinforcement learning," *arXiv preprint arXiv:2604.15414*, 2026.

[17] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1–2, pp. 100–115, 1954.

[18] R. P. Adams and D. J. C. MacKay, "Bayesian online changepoint detection," *arXiv preprint arXiv:0710.3742*, 2007.

[19] B. C. da Silva, E. W. Basso, A. L. C. Bazzan, and P. M. Engel, "Dealing with non-stationary environments using context detection," in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 217–224.

[20] L. N. Alegre, A. L. C. Bazzan, and B. C. da Silva, "Minimum-delay adaptation in non-stationary reinforcement learning via online high-confidence change-point detection," in *Proceedings of the 20th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2021, pp. 97–105.

[21] J. Dick, S. Nath, C. Peridis, E. Benjamin, S. Kolouri, and A. Soltoggio, "Statistical context detection for deep lifelong reinforcement learning," in *Proceedings of the 3rd Conference on Lifelong Learning Agents (CoLLAs)*, 2024.

[22] R. S. Galhardo, P. J. Hastings, and S. M. Rosenberg, "Mutation as a stress response and the regulation of evolvability," *Critical Reviews in Biochemistry and Molecular Biology*, vol. 42, no. 5, pp. 399–435, 2007.

[23] H. G. Cobb, "An investigation into the use of hypermutation as an adaptive operator in genetic algorithms having continuous, time-dependent nonstationary environments," Naval Research Laboratory, Washington, DC, Tech. Rep. AIC-90-001, 1990.

[24] A. E. Eiben, R. Hinterding, and Z. Michalewicz, "Parameter control in evolutionary algorithms," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 2, pp. 124–141, 1999.

[25] J. J. Grefenstette, "Genetic algorithms for changing environments," in *Parallel Problem Solving from Nature 2*, R. Männer and B. Manderick, Eds. North-Holland, 1992, pp. 137–144.

[26] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly, "Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory," *Psychological Review*, vol. 102, no. 3, pp. 419–457, 1995.

[27] C. D. Rosin and R. K. Belew, "New methods for competitive coevolution," *Evolutionary Computation*, vol. 5, no. 1, pp. 1–29, 1997.

[28] J. Branke, "Memory enhanced evolutionary algorithms for changing optimization problems," in *Proceedings of the 1999 Congress on Evolutionary Computation (CEC)*, vol. 3, 1999, pp. 1875–1882.

[29] E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.

[30] M. Herbster and M. K. Warmuth, "Tracking the best expert," *Machine Learning*, vol. 32, no. 2, pp. 151–178, 1998.

[31] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1999.

[32] R. R. Coifman and M. V. Wickerhauser, "Entropy-based algorithms for best basis selection," *IEEE Transactions on Information Theory*, vol. 38, no. 2, pp. 713–718, 1992.

[33] S. G. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, no. 7, pp. 674–693, 1989.

[34] T. Kijewski and A. Kareem, "Wavelet transforms for system identification in civil engineering," *Computer-Aided Civil and Infrastructure Engineering*, vol. 18, no. 5, pp. 339–355, 2003.

[35] Z. K. Peng and F. L. Chu, "Application of the wavelet transform in machine condition monitoring and fault diagnostics: A review with bibliography," *Mechanical Systems and Signal Processing*, vol. 18, no. 2, pp. 199–221, 2004.

[36] W. Zhou, L. Pinto, and A. Gupta, "Environment probing interaction policies," in *International Conference on Learning Representations*, 2019.

[37] G. R. Lee, R. Gommers, F. Waselewski, K. Wohlfahrt, and A. O'Leary, "PyWavelets: A Python package for wavelet analysis," *Journal of Open Source Software*, vol. 4, no. 36, p. 1237, 2019.

[38] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[39] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 5, pp. 834–846, 1983.

[40] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.

[41] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[42] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[43] P. Christodoulou, "Soft actor-critic for discrete action settings," *arXiv preprint arXiv:1910.07207*, 2019.

[44] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1861–1870.

[45] I. Daubechies, "Orthonormal bases of compactly supported wavelets," *Communications on Pure and Applied Mathematics*, vol. 41, no. 7, pp. 909–996, 1988.

APPENDIX A
INFORMATION AVAILABLE TO EACH AGENT

A comparison across agent families is only fair if no agent learns more about the environment than the others. This appendix lists everything each agent receives.

A. The interface

Every agent, ENA and the baselines alike, interacts with an environment only through the Gymnasium interface. A reset returns an initial observation. A step returns the next observation, the reward, and whether the episode terminated or was truncated by the time limit. In CartPole the observation is the cart's position and velocity and the pole's angle and angular velocity. The physical parameters that distinguish the

environments are never part of the observation. In CartPole these are gravity, the pole's mass and the pole's length; in HalfCheetah, gravity, friction and wind. No agent is told when the environment changes, which environment it is in, or how many environments there are. The environment names in our figures and tables come from the experiment's logs and are used only for analysis; ENA's niches are its own numbering.

B. Task constants

ENA reads two constants from the task definition: the episode time limit (500 steps in CartPole) and the solved threshold ($R_{\text{solved}} = 475$). It uses them to rescale a truncated candidate return to a full-episode equivalent (4) and as the utility scale of the specialist selection (8). Both are public properties of the task, not of the physics; the baselines meet the same time limit through the truncation signal.

C. Environment detection

The detector's input is the sequence of transitions (s_t, a_t, s_{t+1}): the observation before acting, the action taken and the observation after, which is exactly what the agent sees. It does not read the reward, the physical parameters or the environment's name. A change of physics changes how the observed velocities evolve, so a niche's dynamics model (5) stops predicting the observations within a few steps. This is why changes between the training environments are detected almost immediately and almost without error. Recognition is weaker in the cases this appendix makes explicit:

- *Physics never seen in training.* At test the detector is frozen and creates no niches. When no niche explains the transitions within the margin m, the episode is left unrecognized and no niche is assigned. The selection (8) still weighs every niche by how well its dynamics model explains the episode, so it falls back on the closest known physics. Physics close to a known niche are recognized as that niche. Earth's and Neptune's gravity, for instance, share one.
- *False alarms.* In a stationary test phase every confirmed change is a false alarm. In physics near the margin m, the detector can alternate between recognizing a niche and leaving the physics unrecognized. Each confirmation resets the selection belief to uniform.
- *Fragmentation.* In some replicates one training environment is split over two niches.

Table X gives these numbers for the two ablation variants with environment detection. Every phase but the run's first follows a real switch, so at most one confirmed change per phase is a detection and the rest are false alarms. The first training phase, Mars, starts with no niche at all.

On HalfCheetah, ENA detects with its chirp (Section III-F5). The chirp's input is the agent's own fixed action sequence and the observations it produces after a reset of the environment, the same observations the agent sees: the chirp reads no reward, physical parameter or environment name either. Its steps count in ENA's step budget, and the environment's random stream is restored after it, so every agent's episodes start from the same states.

TABLE X
ENVIRONMENT RECOGNITION PER PHASE IN THE SKILL ABLATION (30 REPLICATES): UNRECOGNIZED EPISODES, CONFIRMED CHANGES, EPISODES INTO THE PHASE OF THE FIRST ONE, AND HALL OF FAME NICHES PER TRAINING ENVIRONMENT

Agent	Phase	Env.	Unrec. (%)	Changes	First (ep.)	Niches
ENA-HoF+EnvDetection	Training	Mars	0.4 ± 0.4	1.3 ± 1.9	243.0 ± 138.7	1.6 ± 0.6
ENA-HoF+EnvDetection	Training	Jupiter	0.2 ± 0.0	1.1 ± 0.3	1.0 ± 1.0	1.1 ± 0.3
ENA-HoF+EnvDetection	Training	Earth	0.1 ± 0.2	1.2 ± 0.4	0.5 ± 0.7	1.2 ± 0.4
ENA-HoF+EnvDetection	Test	Mars	9.8 ± 24.3	2.0 ± 10.6	6.3 ± 4.2	–
ENA-HoF+EnvDetection	Test	Neptune	1.9 ± 5.7	2.4 ± 5.8	0.2 ± 0.5	–
ENA-HoF+EnvDetection	Test	Jupiter	0.8 ± 0.8	1.0 ± 0.0	0.2 ± 0.4	–
ENA-HoF+EnvDetection	Test	Brown	96.5 ± 11.4	6.8 ± 11.3	0.2 ± 0.5	–
ENA-HoF+EnvDetection	Test	Earth	1.0 ± 0.9	0.0 ± 0.0	–	–
ENA-Combined	Training	Mars	0.3 ± 0.3	1.3 ± 2.5	202.4 ± 174.3	1.4 ± 0.5
ENA-Combined	Training	Jupiter	0.2 ± 0.1	1.1 ± 0.4	0.8 ± 0.9	1.1 ± 0.4
ENA-Combined	Training	Earth	0.0 ± 0.0	1.3 ± 0.5	0.0 ± 0.0	1.3 ± 0.5
ENA-Combined	Test	Mars	1.1 ± 1.1	0.8 ± 1.7	29.5 ± 68.8	–
ENA-Combined	Test	Neptune	0.8 ± 1.1	1.2 ± 1.1	0.3 ± 0.5	–
ENA-Combined	Test	Jupiter	0.6 ± 0.3	1.0 ± 0.0	0.2 ± 0.4	–
ENA-Combined	Test	Brown	92.6 ± 12.6	16.4 ± 18.8	0.0 ± 0.0	–
ENA-Combined	Test	Earth	0.4 ± 0.2	0.0 ± 0.0	–	–

D. Candidate evaluation

ENA evaluates candidate specialists in fresh instances of the current environment, built by the same environment factory the experiment uses. It uses no physical parameter for any decision. Every candidate step is counted in ENA's interaction budget, so a candidate evaluation is equivalent to an extra episode in the current environment, with the same reset any episodic agent has. The baselines could spend the same budget in parallel copies of the environment, as vectorized training does. What ENA does assume is the ability to reset the environment and to run episodes that are not the one being played. That is free in simulation and holds for any episodic task, but it would cost real time on a physical system.

E. Evaluation and test

The first-solve checks (Section IV) run on a separate copy of the current environment, for every agent alike, and their steps are not counted as training. At test, every agent is frozen: the baselines act greedily, and ENA's weights, its Plasticity ($\mathcal{P} = 0$) and its dynamics models are fixed. The only thing that still adapts is the choice of gladiator. Every agent of a replicate meets the same seeded environments.

APPENDIX B
HYPERPARAMETERS

All settings are read directly from the study scripts by `src/tools/paper_config.py`; the values here cannot drift from the code.

TABLE XI
ENA HYPERPARAMETERS (CARTPOLE STUDIES). THESE ARE THE SETTINGS OF ENA-COMBINED AS BUILT BY `CARTPOLE_ENA_ABLATION.PY`; ALL SIX ABLATION VARIANTS SHARE THEM, WITH THE RELEVANT SKILLS SWITCHED ON OR OFF.

Parameter	Value	Parameter	Value
Population size	100	Eval depth (max steps)	500
Return history	100	Exploration ε	0.2
Mutation rate	0.05	Mutation noise σ	0.1
Crossover parent share	0.5	Elite (% of pop.)	20
Mutants (% of pop.)	32	Crossovers (% of pop.)	32
HoF K (max per niche)	5	HoF reinject ratio	0.5
HoF similarity threshold	0.95	HoF gate z	1.5
HoF gate trust floor	0.5	HoF gate relief	0.5
Detection tolerance τ	3	Detection margin m	2
Detection recent window	500	Min. samples to fit	100
Verification samples	50	CuSum threshold $\log \gamma$	10^8
BPR return noise	0.2	BPR switch rate	0.05
BPR prior strength	1	Plasticity controller	linear ramp
Signal warm-up (returns)	10	Signal percentile	90th

TABLE XII
ENA HYPERPARAMETERS THAT DIFFER FROM THE CARTPOLE SETTINGS (`HALF_CHEETAH.PY`). THE BRAIN IS A TWO-LAYER TANH NETWORK WITH NORMC INITIALISATION; ALL OTHER ENA SETTINGS ARE AS IN TABLE XI.

Parameter	Value	Parameter	Value
Brain	(32, 16) tanh	Eval depth (max steps)	200
Mutation rate	0.25	HoF K (max per niche)	10
HoF driver	Olympic	Olympics interval (steps)	200,000
Olympics episodes	3	Olympics qualifier seats	2
Environment detection	chirp	Chirp steps	64
Chirp excitation	wavelet train (Mexican hat)	Chirp transform	wavelet packet
Chirp wavelet	Haar	Chirp level / bands	3 / 8
Chirp window (episodes)	4	Chirp min chirps	8
Chirp confirm margin	3		

TABLE XIII
DQN HYPERPARAMETERS (CARTPOLE COMPARISON). NETWORK: 64×64 RELU MLP.

Parameter	Value	Parameter	Value
Learning rate	3×10^{-4}	Discount γ	0.99
Batch size	64	Replay buffer	50,000
ε start / end	1.0 / 0.01	ε decay steps	10,000
Target network update	every 1,000 steps		

TABLE XIV
PPO HYPERPARAMETERS (CARTPOLE COMPARISON). NETWORKS: SEPARATE 64×64 TANH ACTOR AND CRITIC WITH ORTHOGONAL INITIALISATION.

Parameter	Value	Parameter	Value
Entropy coefficient	0.01	Learning rate anneal	none

TABLE XV
SAC-DISCRETE HYPERPARAMETERS (CARTPOLE COMPARISON). NETWORK: 64×64 RELU MLP. TARGET ENTROPY RATIO $0.5 \times \log|\mathcal{A}|$; STABLE-BASELINES3 DEFAULTS OTHERWISE.

Parameter	Value
Target entropy ratio	$0.5 \times \log\lvert\mathcal{A}\rvert$

TABLE XVI
PPO AND SAC HYPERPARAMETERS (HALFCHEETAH COMPARISON), FROM RL BASELINES3 ZOO (RAFFIN, 2020). NETWORKS: 256×256 RELU MLP (SEPARATE ACTOR AND CRITIC FOR BOTH). DEVIATIONS FROM THE ZOO'S SETTINGS ARE NOTED IN THE SOURCE (`HALF_CHEETAH.PY`).

Parameter	PPO	SAC
Learning rate	2.0633×10^{-5}	3×10^{-4}
Discount γ	0.98	0.99
Batch size	64	256
n-steps (PPO) / buffer capacity (SAC)	512	1,000,000
Update epochs (PPO) / gradient steps (SAC)	20	20 every 20 steps
GAE λ	0.92	—
Clip ratio	0.1	—
Entropy coef. / auto-tune α	4.0×10^{-4}	automatic ($-\dim\mathcal{A}$)
Value coef. / τ	0.58096	0.005
Max grad norm	0.8	—
Initial log std	-2.0	—
Reward scaling	return std	none
Learning starts	—	10,000

ENA — CartPole

ENA — HalfCheetah

DQN (CartPole)

PPO (CartPole)

SAC-Discrete (CartPole)

PPO and SAC (HalfCheetah)

APPENDIX C
COMPUTE ENVIRONMENT

CartPole experiments (all studies except HalfCheetah)

All CartPole studies ran inside a Docker container (`hub/python/Dockerfile`) on the host machine:

- **Host:** Apple MacBook Pro, Apple M1 Pro chip, 8 cores (6 performance + 2 efficiency), 16 GB RAM.
- **Container OS:** Linux (ARM64, via Docker Desktop on macOS).
- **Python:** 3.11.16.
- **Key libraries (pinned):** TensorFlow 2.21.0, Gymnasium[MuJoCo] 1.3.0, MuJoCo 3.13.0, NumPy 2.4.6, Keras 3.15.1, SciPy 1.17.1, PyWavelets 1.8.0. Full list in `app/requirements.txt`.
- **Seeds:** replicate i (1-indexed) seeds Python's, NumPy's and TensorFlow's RNGs with $1000 + i$ before each agent is built; environments are seeded from the same value so all agents of a replicate start from the same random state.

HalfCheetah experiment (cloud run)

The HalfCheetah comparison ran in AWS (eu-west-1), one **c8g.xlarge** instance per replicate (20 instances total):[1]

- **Instance:** AWS c8g.xlarge — 4 vCPU (AWS Graviton4, ARM64), up to 6.25 GBit/s EBS bandwidth, 30 GB root volume.
- **Container OS:** Linux aarch64 (ARM64 Docker image, `hub/cloud/Dockerfile`), 4 OS cores available.

[1]Instance type defined in `infra/src/qos/qos.json`.

- **Python and libraries:** identical pinned versions to the CartPole environment (same `requirements.txt` baked into the image at build time).
- **ENA threads:** 2 (candidate evaluation runs on 2 threads on the 4-core instance).
- **Seeds:** replicate i (1-indexed) seeds Python's, NumPy's and TensorFlow's RNGs with i before each agent is built; the HalfCheetah study does not use a seed offset.
- **Wall-clock time per replicate:** 32.0 min (ENA), 115.3 min (PPO), 281.1 min (SAC).

APPENDIX D
EXTENDED CONTINUOUS-CONTROL DYNAMICS

This appendix presents the complete episode-by-episode evaluation curves and step-by-step training trajectories for the continuous-control HalfCheetah experiment across all 20 confirmation replicates.

A. Frozen Test Dynamics Across Environments

Fig. 6 illustrates the test phase across the sequential protocol of five environments (50 evaluation episodes per environment): Earth-Standard $\rightarrow$ Ice-Floor $\rightarrow$ Heavy-Rough $\rightarrow$ High-Wind $\rightarrow$ Mars-Slippery. Solid lines show the mean test return across all 20 replicates, and shaded bands mark the middle 50% interquartile range.

On *Heavy-Rough* (episodes 101–150), both PPO and SAC suffer catastrophic performance collapse: after having adapted to Mars-Slippery during the final training phase, their frozen policies drop to mean returns of 1301.8 and 1301.1, respectively. In contrast, ENA-Combined identifies the environment switch via its wavelet-packet chirp and successfully retrieves its archived Heavy-Rough specialist, sustaining a mean return of 2417.1.

On the two environments never seen during training (*Ice-Floor* and *High-Wind*), all agents operate zero-shot. SAC's continuous representations achieve high baseline locomotor velocity on Ice-Floor, whereas ENA's chirp identifies these novel physics as unrecognized, falling back on its best-matching archived specialist without catastrophic instability.

B. Training Dynamics Across Sequential Environments

Fig. 7 displays the training trajectories of all three agents over the full interaction budget of 3,000,000 environment steps (1,000,000 steps per training phase: Earth-Standard $\rightarrow$ Heavy-Rough $\rightarrow$ Mars-Slippery). Returns are binned and smoothed over step intervals across the 20 replicates.

During training, SAC's off-policy gradient optimization rapidly masters each individual environment, climbing to peak returns of $\sim$8,500 on Earth-Standard, $\sim$7,500 on Heavy-Rough, and $\sim$9,500 on Mars-Slippery. However, the sharp discrepancy between SAC's high training return at step 2.0×10^6 on Heavy-Rough and its test return of 1301.1 in Fig. 6 visually demonstrates the severity of catastrophic forgetting when weights are continuously updated without specialist compartmentalization.

ENA-Combined makes steady progress within its interaction budget, achieving end-of-phase training averages of 1,805 on Earth-Standard and 2,133 on Heavy-Rough. Because ENA's step budget is shared between exploratory mutation evaluations, chirp excitation sequences, and active policy execution, its end-of-phase training scores reflect active search variance; when evaluated with frozen elites at test time, it maintains and exceeds these levels (2000.1 on Earth-Standard, 2417.1 on Heavy-Rough).

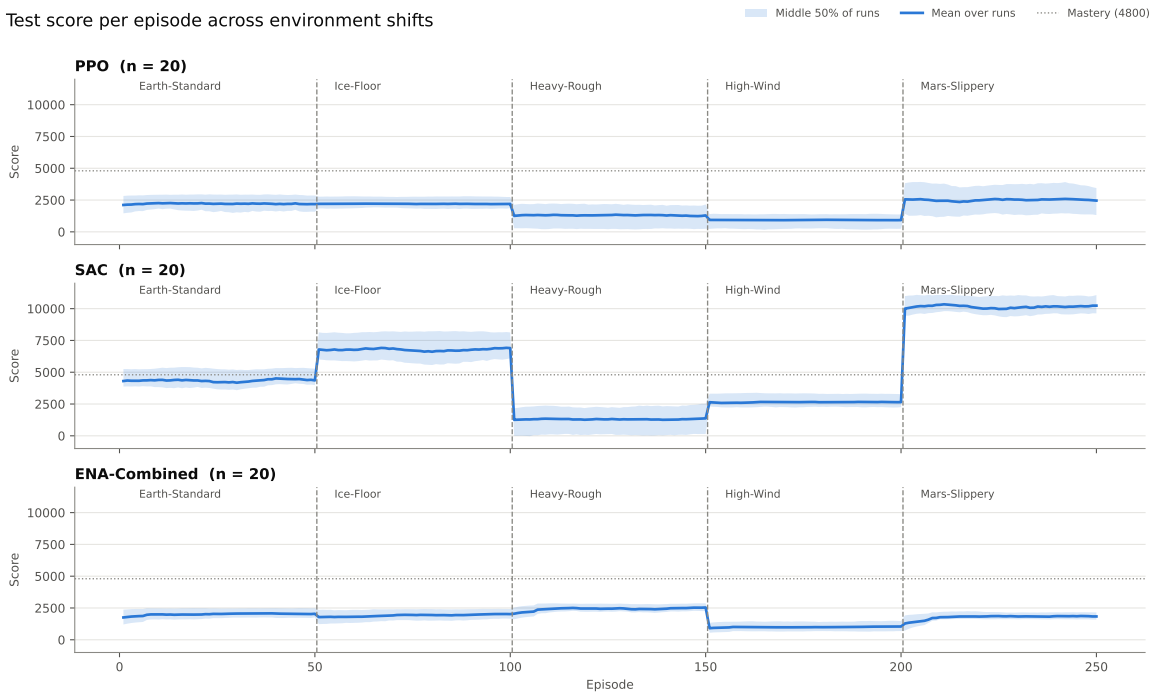


Fig. 6. Frozen test on HalfCheetah: mean return per episode across 20 confirmation replicates (line) and middle 50% interquartile range (band). Dashed vertical lines mark environment transitions; Ice-Floor and High-Wind were never encountered during training. While SAC and PPO suffer severe forgetting on Heavy-Rough, ENA-Combined recalls its archived specialist.

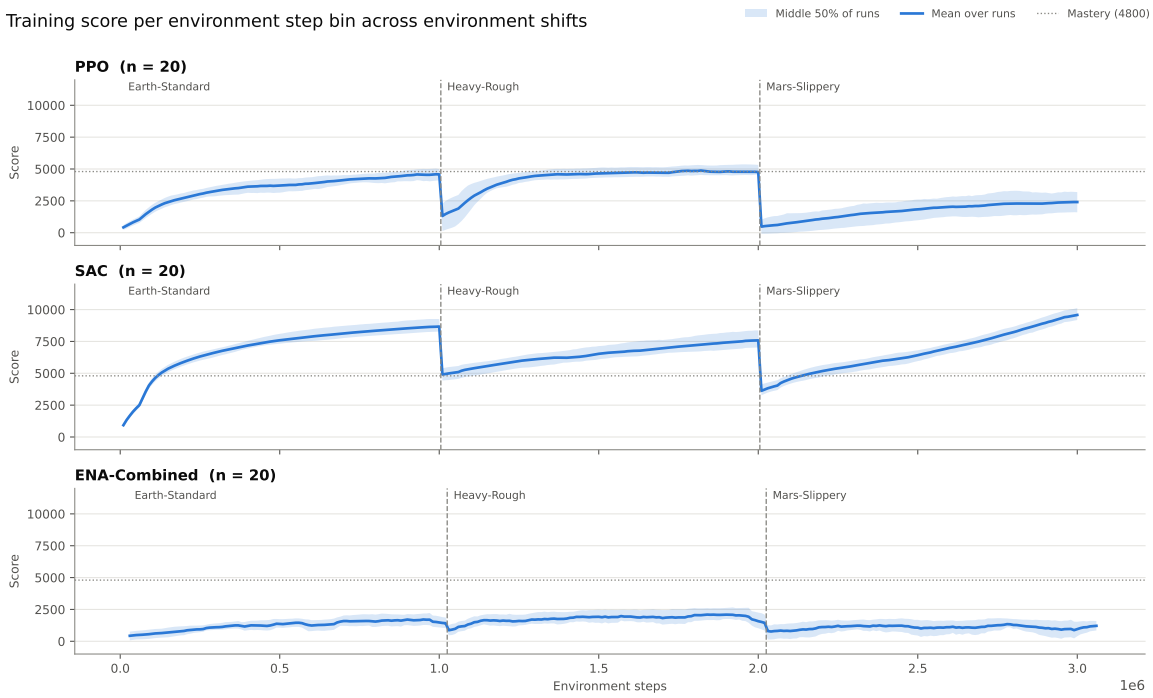


Fig. 7. Training score binned across environment steps over the 3,000,000-step budget on HalfCheetah (20 replicates; line is mean, band is middle 50% of replicates). Dashed vertical lines mark transitions between the three sequential training environments (Earth-Standard → Heavy-Rough → Mars-Slippery).